

QUANTITATIVE RESEARCH PORTFOLIO

Design and Implementation of an Evolutionary Walk-Forward Research Framework for Systematic Intraday Trading

A Nasdaq-100 E-mini (NQ) five-day development experiment using 4-tick range bars: genetic-algorithm optimisation, walk-forward validation, trade diagnostics, and Monte Carlo robustness analysis.

Alexandre Kupfermunz

Quantitative Research Portfolio Project · June–July 2026

Table of Contents

Table of Contents.....	2
Executive Summary	4
1. Research Problem and Objectives.....	5
The problem.....	5
Objectives.....	5
Research questions.....	5
Scope	5
2. Framework Design	6
Design principles.....	6
Engineering value.....	6
3. Research Pipeline: Strategy and Backtester	7
Data processing.....	7
Strategy generation	7
Backtesting and metrics	7
Why this structure matters	8
4. Research Pipeline: Optimisation and Validation	9
Genetic algorithm	9
Fitness functions	9
Rolling vs. Expanding: What Each Design Actually Tests.....	10
Automated outputs	10
5. Experimental Setup and Reproducibility	11
Experiment management	11
Experiment configuration.....	12
Reproducibility controls already in place	12
Reproducibility gap.....	12
6. Configuration-Level Results.....	13
7. Genetic Algorithm Learning and Generalisation	14
Interpretation	14
8. Trade-Level Diagnostics	15
9. Stability, Directionality and Profit Concentration	16
Direction breakdown	16
Profit concentration.....	16
Window and parameter stability	16
What this diagnostic prevents.....	16

10. Monte Carlo Robustness Analysis	17
Other results	18
11. Monte Carlo Distributions and Interpretation	19
What the simulation can answer	19
What it cannot answer	19
12. Limitations and Validity Risks.....	20
Data limitations	20
Computational limitations.....	20
Methodological limitations	20
Implementation and reporting risks	20
13. Future Work and Conclusion	21
Software development.....	21
Quantitative research.....	21
Main things learned.....	21
Conclusion	21

Executive Summary

Good quantitative research requires far more than a profitable-looking trading rule. It requires repeatable data processing, honest optimisation, genuine out-of-sample testing, robustness checks, and well-organised experiment records. This project was built to learn and apply that complete process, using a short-term intraday strategy as the vehicle rather than the end goal.

The case study uses five days of high-frequency Nasdaq-100 E-mini futures (NQ) data, represented as 4-tick range bars. Training and out-of-sample testing were restricted to the 14:30–17:30 (Europe/Brussels) trading window, and commission and slippage assumptions were applied consistently throughout optimisation and analysis.

The experiment covers eight configurations, three stochastic repetitions per configuration, and 78 out-of-sample windows, producing 297 CSV files and 123 plots. The best-ranked configuration — expanding walk-forward validation with a losing-streak fitness function — produced pooled out-of-sample results close to break-even: \$30 net profit over 135 trades, a profit factor of 1.016, and a maximum drawdown of \$385.

The more important finding is not this small positive figure. The results reveal substantial variation between repetitions, an overfitting diagnostic triggered on 47 of 78 windows, a thin trade sample, and a wide Monte Carlo distribution once resampling uncertainty is accounted for: 48.7% of the 5,000 bootstrap simulations finished with a negative net profit. The appropriate conclusion is that the framework performs well as research infrastructure, while the strategy evidence itself remains inconclusive.

Main point — The strongest output of this project is the reusable research framework and its ability to surface weak evidence rather than obscure it. The five-day strategy result is a development case study, not a claim of profitability.

1. Research Problem and Objectives

The problem

A simple backtest can look strong for the wrong reasons: an optimiser can fit noise, results can depend on a single favourable market period, and small implementation choices can materially change reported profit. For this reason, the project was not organised around the question “Can I find a profitable strategy?”. The more useful question was: “Can I build a research process that tests strategies rigorously, and that reveals its own weaknesses?”

The impulse strategy used throughout this report is a case study only. It gives the framework a realistic optimisation problem, but it is not the main deliverable — the pipeline that trains, validates, diagnoses, and reports on it is.

Objectives

- Build a modular pipeline in which data handling, signal generation, backtesting, optimisation, and analysis are cleanly separated.
- Run the same strategy under different fitness functions and walk-forward designs without changing strategy code.
- Measure training-set learning, out-of-sample transfer, and optimiser-to-optimiser variance.
- Generate automatic, auditable CSV tables and plots so every experiment can be reviewed later.
- Surface the framework's and the strategy's limitations before making any performance claim.

Research questions

- Can the framework run the full pipeline across many validation settings without modifying the strategy code?
- Does the genetic algorithm reliably improve its training objective across generations?
- Does that training-set improvement transfer to unseen walk-forward windows?
- Can the analysis layer detect instability, overfitting, and risk concentration on its own?

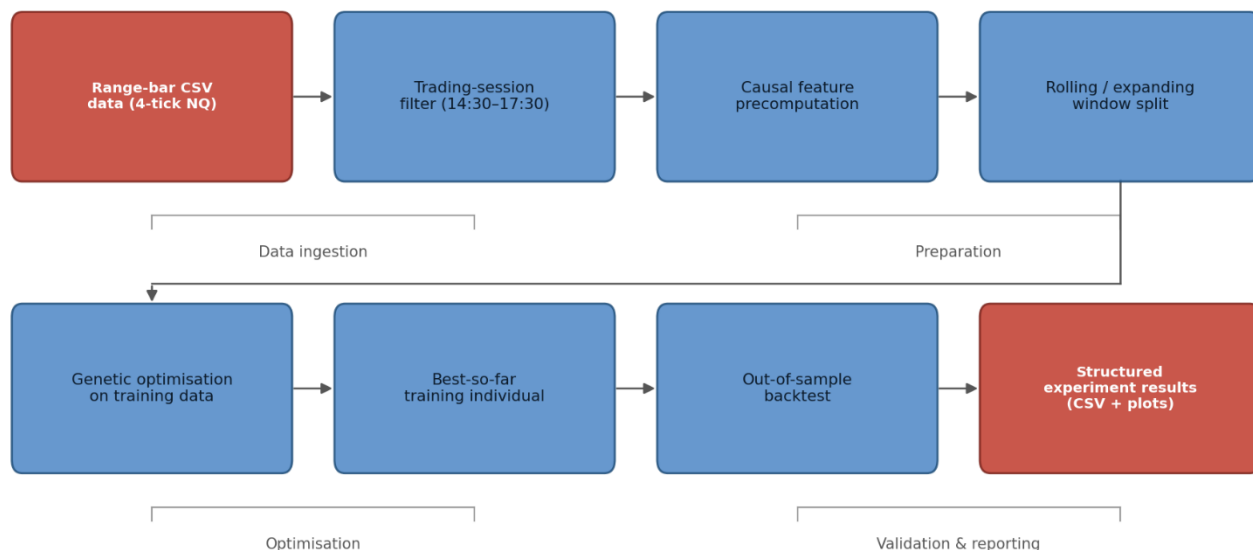
Scope

The experiment uses five trading days of high-frequency NQ data represented as 4-tick range bars. This is a development-scale dataset used to validate the infrastructure and the diagnostic logic, not to establish a durable trading edge; the sample is far too short to support a claim of real profitability.

2. Framework Design

The framework follows a modular design: each stage has a single responsibility and passes structured data to the next. This keeps debugging tractable and allows any one component to be replaced without rewriting the rest of the project.

Research pipeline: from raw data to reported evidence



[Figure 1 — End-to-end research pipeline]

End-to-end research pipeline, from raw range-bar data through optimisation to reported evidence

Design principles

- Clear separation between strategy logic, optimisation, and analysis.
- Experiments driven by configuration objects instead of manual code edits.
- Reusable components intended to support future strategy families, not just this one.

Engineering value

- Consistent folder and file naming conventions across every experiment.
- Local (per-repetition) and global (cross-configuration) diagnostic modules.
- Well-defined extension points for parallelisation, checkpointing, and new fitness objectives.

3. Research Pipeline: Strategy and Backtester

Data processing

The pipeline starts from high-frequency NQ data represented as 4-tick range bars. Unlike fixed-time candles, a range bar closes on price movement rather than on the clock: each bar spans a four-tick price range, equivalent to one NQ index point. Active markets produce more bars per unit of time; quiet markets produce fewer.

Before feature computation and strategy evaluation, the framework applies a trading-session filter. Users may specify one or more non-contiguous trading windows via start and end times; only 14:30–17:30 (Europe/Brussels) was used in this experiment, and the same filter was applied consistently to every training and test window.

Data is split into training and test partitions before optimisation, and features are computed separately within each partition. This ordering reduces the risk of future information leaking into the training process.

Strategy generation

The case study is an impulse-based intraday strategy: it looks for short, rapid directional moves in the range-bar stream. The genetic algorithm does not alter the strategy family — it searches over six parameters that control the impulse-detection and exit rules, which keeps the problem interpretable while still defining a meaningfully large search space.

Parameter	Role in the strategy	Selected range
Minimum impulse candles	Minimum consecutive 4-tick range bars needed to define an impulse	5 to 15
Maximum duration (ms)	Maximum clock time allowed for the full range-bar impulse	200 to 4,000
Imbalance ratio threshold	Minimum diagonal order-flow imbalance required inside the range bar	3.00 to 10.00
Minimum imbalance count	Minimum qualifying imbalances across the impulse bars	1 to 15
Take-profit (ticks)	Profit target used by the backtester	2 to 30
Stop-loss (ticks)	Loss limit used by the backtester	2 to 30

Backtesting and metrics

The backtester simulates the entry and exit of every trade, including commission and fixed slippage. At entry, price is moved one tick against the strategy: a long entry is filled one tick above the signal price; a short entry, one tick below.

Take-profit and stop-loss levels are computed from the executed entry price rather than the signal price. Take-profit is treated as a resting limit order, so no additional slippage applies when it is reached. When the stop-loss is reached, one further tick of adverse slippage is applied to the exit.

Example — A long signal appears at 100. With one tick of entry slippage, the fill is 101. A five-tick take-profit targets 106 with no further slippage. A two-tick stop-loss is nominally at 99, but the simulated exit is 98 once the extra tick of adverse slippage is applied. A winning trade therefore still earns its full target distance from the entry price, while a losing trade loses its stop distance plus one further tick.

Every trade is stored with its entry, exit, direction, source window, selected generation, and result. The metrics layer then computes net profit, gross profit and loss, profit factor, drawdown, win rate, expectancy, and losing-streak length.

Because the input is event-driven, a holding period measured in bars is not a fixed amount of clock time — the same bar count can form quickly in a volatile period or slowly in a quiet one. The strategy therefore also uses an explicit millisecond-duration parameter, letting the optimiser distinguish a genuinely fast price move from the same number of bars forming slowly.

Why this structure matters

Execution assumptions — commission, slippage, and trading sessions — are passed to the backtester through configuration rather than hard-coded into strategy logic, so they can be changed without touching strategy code. The same execution assumptions are used during both optimisation and out-of-sample testing, meaning the optimiser already selects parameters using cost-adjusted results.

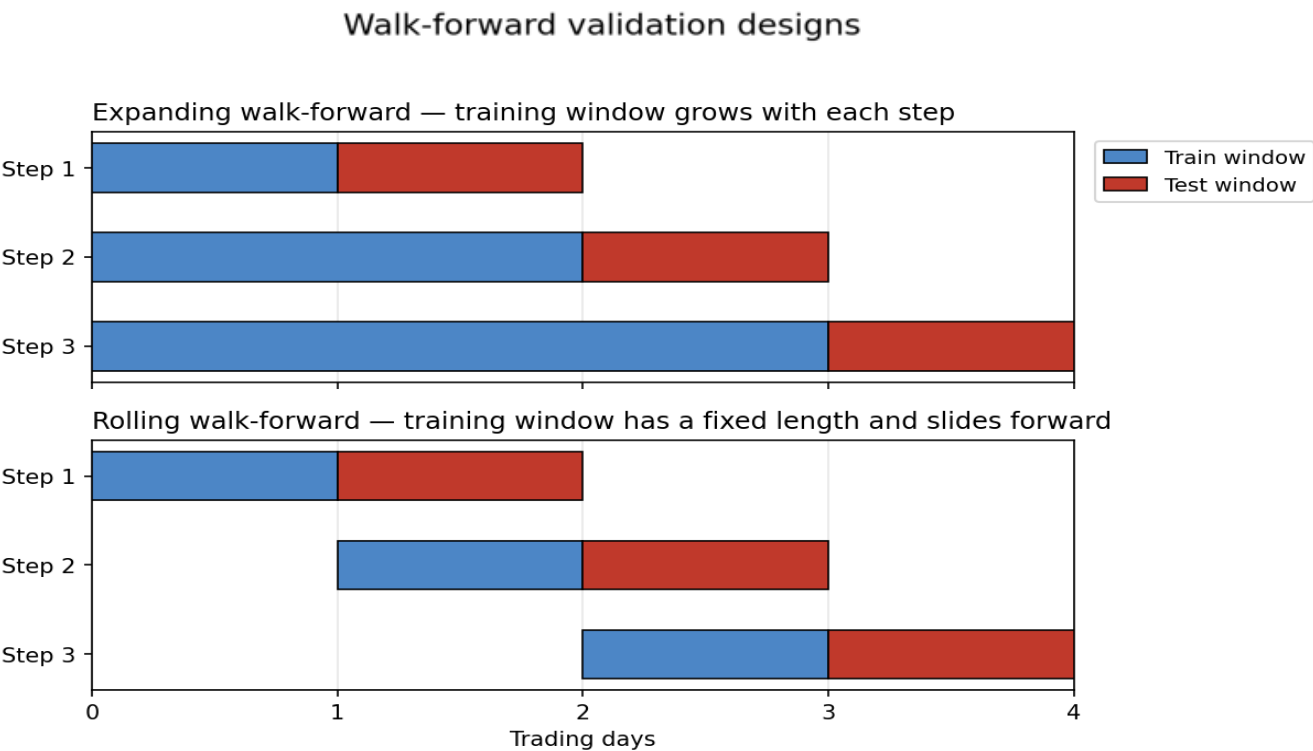
4. Research Pipeline: Optimisation and Validation

Genetic algorithm

Each individual represents one combination of the six strategy parameters. The population evolves through tournament selection, per-gene crossover, and bounded mutation, with generation 0 through generation 100 recorded for every walk-forward window. The framework retains both full population statistics and the best-so-far individual, so the optimisation trajectory can be audited after the fact.

Fitness functions

Two objectives were compared. The net-profit objective rewards direct economic performance. The losing-streak objective adds explicit risk control by penalising long runs of losses, functioning somewhat like a regularising term rather than optimising a single noisy P&L number.



[Figure 2 — Expanding and rolling walk-forward validation]

Expanding walk-forward (training window grows each step) versus rolling walk-forward (training window has a fixed length and slides forward). A test window is only evaluated after parameters are selected on strictly earlier data.

Rolling vs. Expanding: What Each Design Actually Tests

The two designs answer different questions, and this distinction matters for how the results in later sections should be read. Rolling walk-forward, with its fixed training length, is the more direct test of whether a given parameter set generalises to unseen data under comparable conditions from one window to the next.

Expanding walk-forward instead tests something narrower: whether average training fitness keeps improving as the training sample grows — useful for judging whether the optimiser benefits from additional history before committing to a fixed rolling window size, but not, by itself, a like-for-like generalisation test across windows of equal length. In this project, the expanding configuration happened to produce the strongest out-of-sample results of the eight tested (Section 6), so it was the one carried forward into the trade-level and Monte Carlo analysis in Sections 8–11. That choice was made because it was the best-performing configuration available for demonstrating the trade-diagnostic and robustness tooling — not because expanding walk-forward is inherently the right design for answering a generalisation question that rolling walk-forward is better suited to.

Automated outputs

After each run, the experiment manager writes raw results, selected trades, and generation-by-generation histories to disk. Separate analysis modules then generate local repetition diagnostics, global cross-configuration comparisons, window summaries, trade reports, and Monte Carlo outputs — so an experiment can be reproduced and compared without rebuilding any table by hand.

5. Experimental Setup and Reproducibility

Field	Value
Instrument	Nasdaq-100 E-mini futures (NQ)
Dataset	Five-day development sample
Bar representation	4-tick range bars
Trading time zone	Europe/Brussels
Active trading window	14:30 to 17:30
Sampling method	Event-driven, based on price movement rather than fixed time
Nominal price range	Four ticks, equal to one NQ index point
Validation methods	Expanding and rolling walk-forward
Fitness objectives	Losing-streak fitness and net-profit fitness
Rolling training lengths	1 day, 2 days, and 3 days
Repetitions	3 per configuration (24 optimisation runs)
Out-of-sample windows	78 total
Generation records	7,878 population summaries; generations 0 to 100
Monte Carlo	5,000 simulations; random seed 42
Monte Carlo sample size	135 trades per simulation
Simulation input	Net trade results after slippage and commission
Tick value	\$5 per tick
Commission	\$4 per completed trade
Slippage	One tick, unfavourable direction
Take-profit execution	Limit order; no additional exit slippage
Stop-loss execution	One additional tick of unfavourable slippage
Output evidence	297 CSV files and 123 PNG plots

Experiment management

The result archive contains 96 raw result files, 96 global generation-analysis files, 192 local diagnostic files, 12 window-analysis outputs, 12 trade-analysis outputs, and 12 Monte Carlo outputs. The folder structure consistently encodes walk-forward type, fitness function, training size, and repetition.

Experiment configuration

The configuration object defines trading sessions, tick value, commission, slippage rules, and maximum holding period. A single trading session — 14:30 to 17:30 (Europe/Brussels) — was used here, though the framework supports several disjoint sessions within one configuration. The same execution settings feed both fitness evaluation and final analysis: every trade carries one tick of adverse entry slippage, take-profit exits are filled as limit orders with no extra slippage, stop-loss exits carry one additional tick of adverse slippage, and commission is deducted from every completed trade.

Reproducibility controls already in place

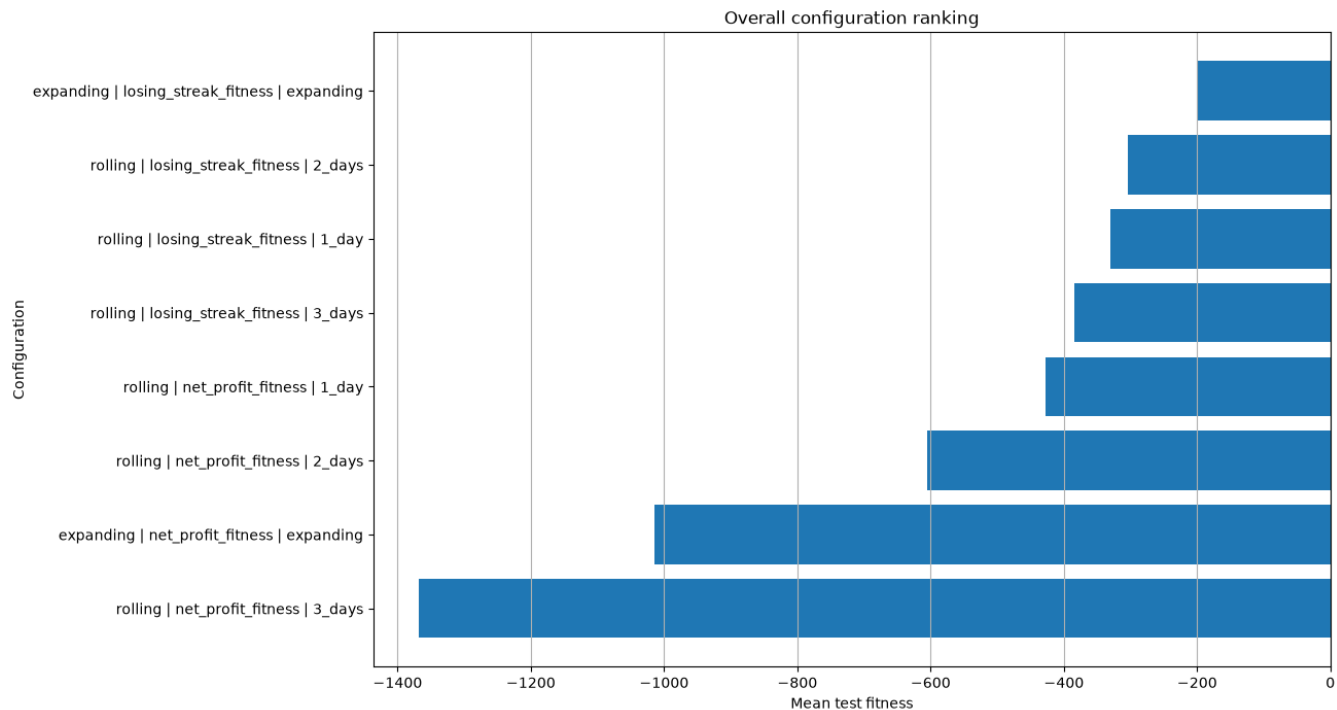
- Named configuration and repetition folders.
- Recorded window boundaries and selected parameters.
- Stored generation-level population statistics.
- Stored trade-level records with source window and repetition.
- Fixed Monte Carlo seed and simulation count.

Reproducibility gap

The archive does not yet include a complete experiment manifest. Missing items include the exact range-bar construction settings, contract month, session definition and daylight-saving handling, end-of-window position treatment, dataset checksum, software environment, and Git commit hash. A future manifest should also record the exact commission and slippage rules alongside each result set.

6. Configuration-Level Results

The configuration ranking is a diagnostic tool, not proof that the top-ranked configuration is tradable: reported test fitness includes a window-size multiplier and a substantial no-trade penalty. Net profit, drawdown, and trade count are therefore reported alongside it. All eight configurations share the same dataset, range-bar construction, trading session, commission, and slippage, so the comparison isolates the effect of the fitness function and the walk-forward design.



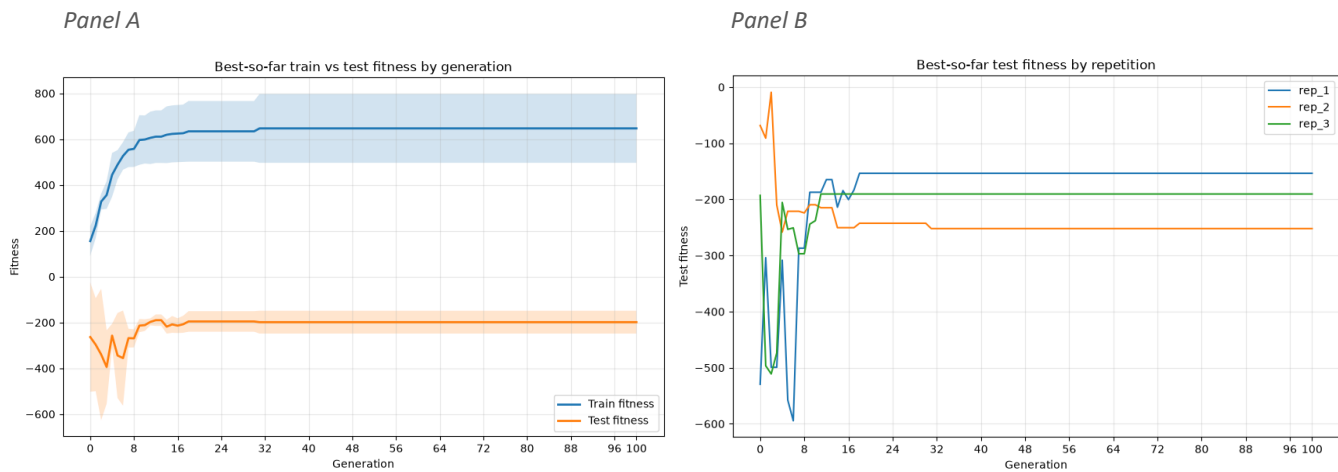
[Figure 3 — Configuration comparison]

Horizontal bar chart ranking all eight configurations by mean test fitness (all values negative; losing-streak configurations rank highest, net-profit configurations lowest).

Rank	Configuration	Mean test fit.	Mean P&L	Mean DD	Trades/window
1	Expanding / Losing-streak	-199	2.5	106.9	11.2
4	Rolling / Losing-streak, 3d	-385	65.2	53.8	5.2
6	Rolling / Net-profit, 2d	-605	30.8	53.6	3.0
7	Expanding / Net-profit	-1014	-59.9	145.6	8.4

The losing-streak objective occupies the top four ranks. Averaged across 39 windows per objective, mean test fitness was -292 for losing-streak versus -794 for net-profit. Both objectives still show negative average test fitness, and the positive per-window P&L averages shown above rest on very few trades.

7. Genetic Algorithm Learning and Generalisation



[Figure 4 — GA learning and test-fitness diagnostics]

Panel A: average best-so-far train fitness versus test fitness by generation for the selected configuration — train fitness rises quickly and plateaus around generation 20–30, while test fitness stays low and noisy before the plateau. Panel B: best-so-far test fitness by generation for the three repetitions (rep_1, rep_2, rep_3), settling at clearly different levels.

Diagnostic	Result
Training learning detected	75 of 78 windows (96.2%)
Overfitting flags	47 of 78 windows (60.3%)
Median selected generation	9
Selected by generation 20	65 of 78 windows (83.3%)
Selected configuration overfit rate	7 of 12 windows (58.3%)

The optimiser reliably learns on the training data: population fitness improves quickly and typically plateaus well before the run ends. The median selected generation is 9, even though 100 generations were recorded for every window — additional generations were rarely useful.

The harder problem is transfer. Test fitness stays noisy and the train–test gap widens as training improves. This pattern is expected when a high-capacity search repeatedly evaluates a small dataset; early stopping should rely on a dedicated validation rule rather than on the final test window itself.

Interpretation

A working optimiser is not the same thing as a working strategy. The framework clearly improves the training objective, but in doing so it also makes the generalisation problem visible rather than hiding it.

8. Trade-Level Diagnostics

Detailed trade and Monte Carlo analysis was carried out on the best-ranked configuration — expanding walk-forward with losing-streak fitness — pooling the out-of-sample trades from its three repetitions.

As noted in Section 4, this configuration was selected for its results rather than because expanding walk-forward is the design best suited to testing generalisation; the trade-level diagnostics below should be read with that in mind.

All P&L, expectancy, profit-factor, and drawdown figures in this section and in Sections 10–11 use one consistent money conversion: slippage is embedded in each trade result, results are converted at a \$5 tick value, and \$4 commission is deducted from every completed trade

Pooled metric	Result
Number of trades	135
Net profit after commission and slippage	\$30
Profit factor	1.016
Win rate	27.4%
Expectancy per trade	\$0.22
Maximum drawdown	\$385
Recovery factor	0.078
Longest losing streak	14

Repetition	Trades	P&L	PF	Win rate	Max DD	Recovery
rep_1	49	\$169	1.28	34.7%	\$162	1.04
rep_2	58	-\$167	0.80	22.4%	\$347	-0.48
rep_3	28	\$28	1.07	25.0%	\$209	0.13

The pooled result is only marginally positive, and the repetitions disagree sharply: +\$169, -\$167, and +\$28. Identical configuration settings can therefore follow very different optimisation paths and produce very different outcomes.

9. Stability, Directionality and Profit Concentration

Direction breakdown

Long trades lost \$247 over 43 trades at a 14.0% win rate. Short trades made \$277 over 92 trades at a 33.7% win rate. The pooled positive result exists only because the short side offsets the long-side loss.

Profit concentration

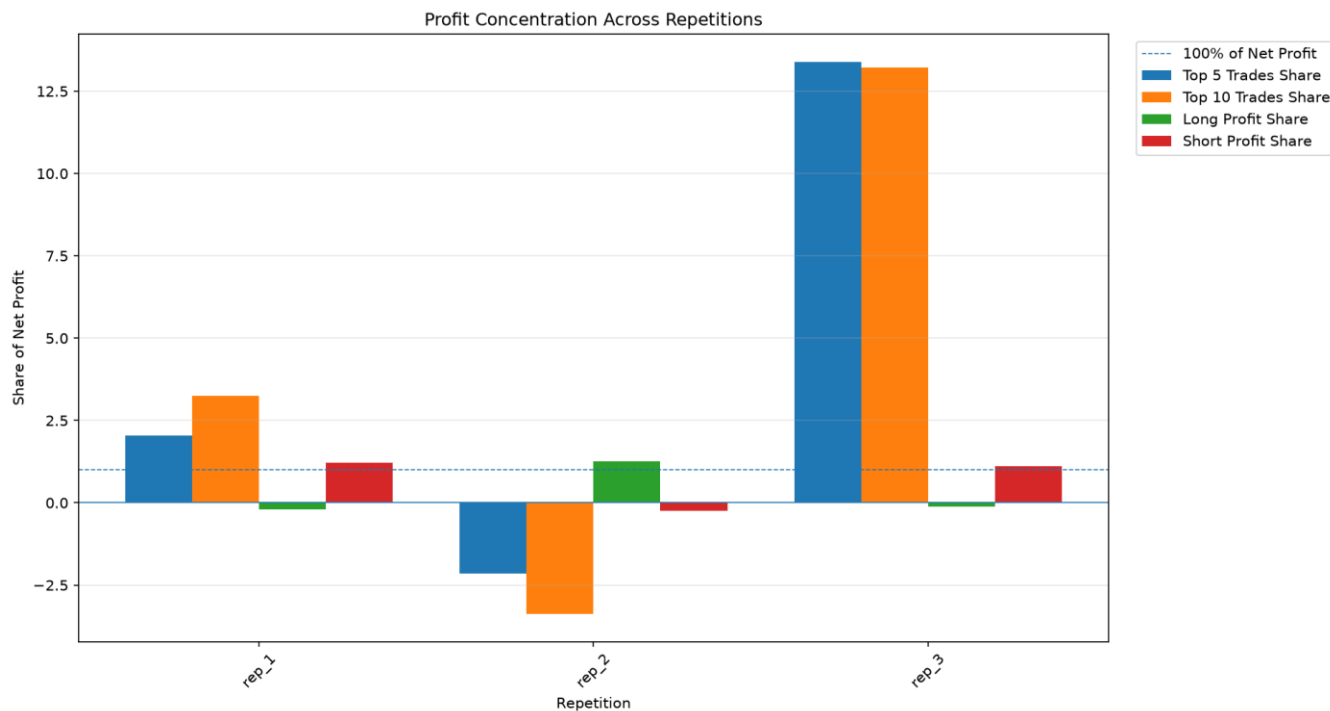
The top five trades contributed \$555 — 18.5 times the final pooled profit. The top ten trades contributed \$905, or 30.2 times the final profit. Most losing trades cluster near the fixed loss of roughly -\$19, while winners show a long right tail.

Window and parameter stability

Average per-window profit was \$10, with a standard deviation of \$235; the worst window lost \$222 and the best gained \$248. The selected stop-loss was two ticks in all 12 selected windows, while every other parameter kept changing — suggesting one stable local preference rather than a stable overall parameter set.

What this diagnostic prevents

Without trade-level analysis, the pooled +\$30 result could look mildly encouraging in isolation. The breakdown shows it depends on one side of the book, a handful of large winners, and one favourable repetition.



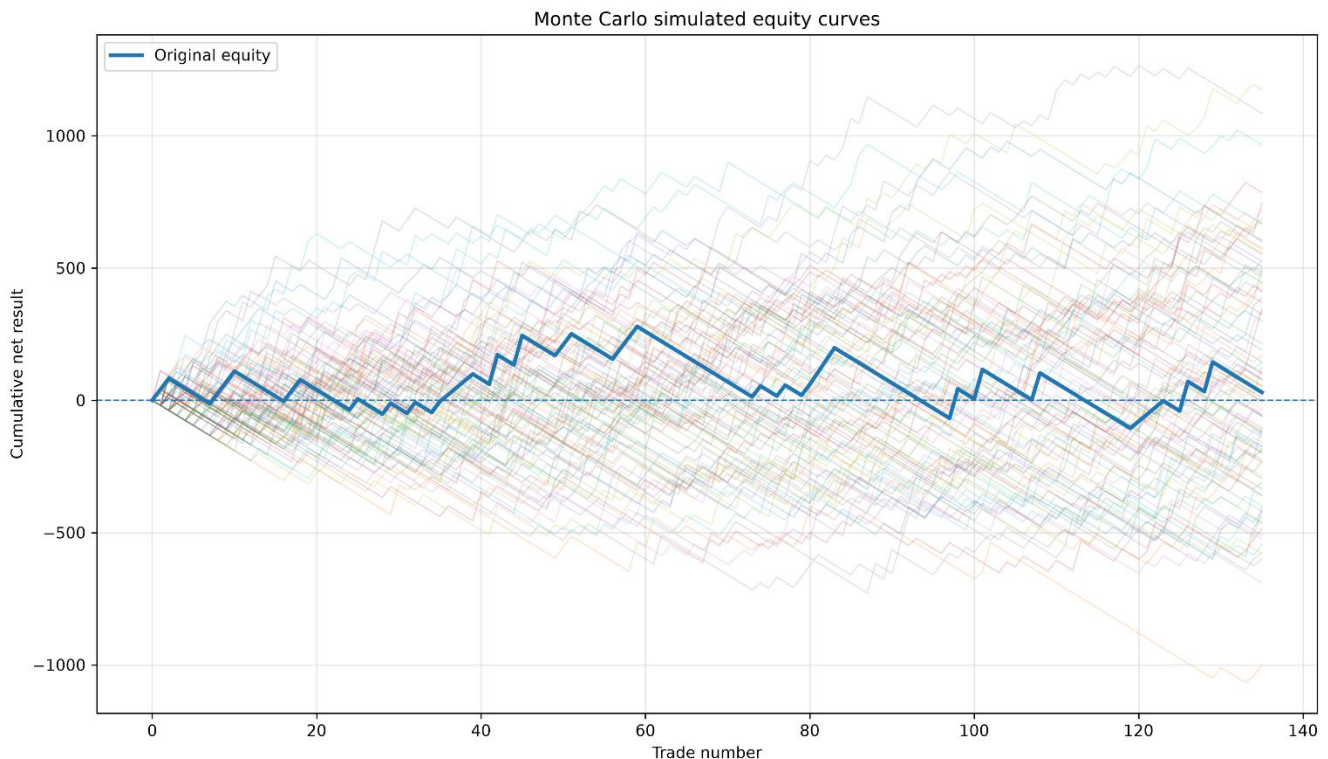
[Figure 5 — Trade-result distribution and profit concentration]

Grouped bar chart per repetition showing top-5 trade share, top-10 trade share, long-side profit share, and short-side profit share.

10. Monte Carlo Robustness Analysis

The Monte Carlo module resamples the 135 out-of-sample trades from the best configuration (expanding walk-forward, losing-streak fitness). Each trade already includes backtester slippage; results are converted from ticks to dollars using the \$5 tick value, with \$4 commission deducted from every completed trade — the same conversion used throughout Sections 8–9.

The module runs 5,000 bootstrap simulations with a fixed random seed of 42. Each simulation draws 135 trades with replacement from the observed sample, so a given trade can appear multiple times in one simulation or not at all. This resampling changes both the order and the composition of the trade sample — unlike a pure reshuffle, which would only change order and leave every simulation's final profit identical. Sampling with replacement instead produces a genuine distribution of possible outcomes.



[Figure 6 — Monte Carlo simulated equity paths]

Cumulative net result against trade number: many faint simulated paths fan out from roughly -\$1,000 to +\$1,500, with the original out-of-sample equity curve overlaid as a bold line staying close to zero.

Metric	Original	Median	P5	P95
Net profit	\$30	\$15	-\$605	\$715
Maximum drawdown	\$385	\$412	\$210	\$798
Profit factor	1.016	1.008	0.698	1.415
Win rate	27.41%	27.41%	21.48%	34.07%
Expectancy per trade	\$0.22	\$0.11	-\$4.48	\$5.30
Recovery factor	0.078	0.033	-0.874	2.970
Longest losing streak	14	12	8	20

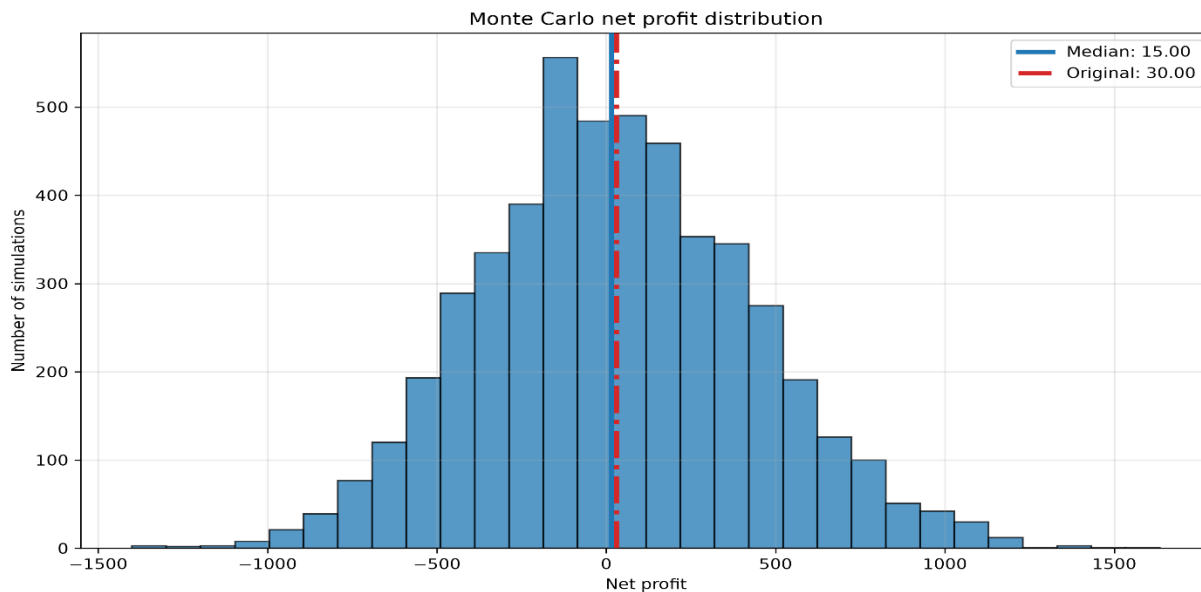
Other results

- Mean net profit: \$28.41; standard deviation of net profit: \$401.28.
- Negative simulations: 2,433 of 5,000 (48.7%).
- Simulations with drawdown above the original \$385: 56.4%.
- Simulated net profit range: -\$1,400 to \$1,635.

The original net profit of \$30 sits close to both the simulated mean (\$28.41) and median (\$15), so the observed out-of-sample result is not unusual within the bootstrap distribution — but that distribution is very wide: its standard deviation of \$401.28 dwarfs the profit itself. A negative outcome occurred in 48.7% of simulations, and the 5th–95th percentile range spans -\$605 to \$715, meaning positive and negative final results are close to equally likely.

The original maximum drawdown of \$385 is slightly below the simulated median of \$412, and in 56.4% of simulations drawdown exceeded the original figure — so the realised path was not an unusually bad one, but materially larger drawdowns remain clearly possible. The original profit factor of 1.016 sits close to the simulated median of 1.008, while the 5th-percentile profit factor of 0.698 corresponds to a clear losing outcome, confirming that this trade sample sits close to break-even with little protection against sampling variation.

11. Monte Carlo Distributions and Interpretation



[Figure 7 — Monte Carlo net-profit and drawdown distributions]

Left: distribution of simulated net profit, roughly bell-shaped and wide, with markers at the median (\$15) and the original (\$30). Right: distribution of simulated maximum drawdown, right-skewed, with markers at the median (\$412) and the original (\$385).

What the simulation can answer

- How trade ordering affects the shape of the equity path.
- The distribution of net profit and drawdown under resampling of the observed trades.
- Plausible losing-streak lengths given the observed trade sample.
- Whether the realised sequence was unusually good or unusually bad relative to that distribution.

What it cannot answer

- Whether the strategy has genuine predictive power.
- Model-selection bias introduced by choosing the best of eight configurations.
- Performance under volatility or liquidity regimes outside these five days.
- Serial dependence between trades, changing fill quality, or market impact.

The simulation is centred close to the observed result, but this does not validate the strategy: the resampled trades were already selected after optimisation and configuration ranking, so the Monte Carlo analysis measures path uncertainty conditional on that selection. It does not capture optimiser uncertainty, model-selection uncertainty, or regime uncertainty. A stronger future test would re-run the full optimisation inside each resampling loop, or evaluate a much larger, untouched period after all model choices are frozen.

Robustness conclusion — The Monte Carlo distribution is centred near the observed result, but almost half of the simulations finish below zero and large drawdowns remain plausible. This should be read as evidence of strategy fragility, not as evidence of a lasting edge.

12. Limitations and Validity Risks

Data limitations

- Five-day sample spanning a limited range of market regimes.
- Single market and single strategy family.
- Few trades overall, including 23 no-trade windows.

Computational limitations

- Fitness evaluation is CPU-intensive.
- The backtester is not fully vectorised or JIT-compiled.
- Parallel, checkpointed, and distributed execution need further work.

Methodological limitations

- Only three stochastic repetitions per configuration.
- Eight configurations selected and compared on the same development sample.
- No nested validation layer and no untouched final test set.
- No placebo baseline and no transaction-cost sensitivity study yet.

Implementation and reporting risks

- The test-fitness multiplier and no-trade penalty affect the configuration ranking and should be read as a diagnostic, not a P&L estimate.
- Contract details, session definition, and range-bar construction settings are not yet captured in the experiment archive.
- No Git commit hash, software environment record, or dataset checksum is stored alongside results.

Validity statement — The current evidence is sufficient to evaluate the software and the research workflow. It is not sufficient to evaluate the strategy's economic viability.

13. Future Work and Conclusion

Software development

- Parallel evaluation and profiling of expensive functions.
- NumPy vectorisation and Numba/JIT compilation where it matters most.
- Checkpointing, resume support, and configuration validation.

Quantitative research

- Move from five days to one year, then to several years and market regimes.
- Run 20–30 GA seeds and report full optimiser distributions rather than point estimates.
- Introduce train, validation, and untouched final-test layers.
- Test additional markets and a range of execution-cost scenarios.

Main things learned

- Training-set improvement does not guarantee out-of-sample performance.
- Repeated runs and trade-level decomposition are necessary before an equity curve can be trusted.
- The research infrastructure is more valuable, and more durable, than any single five-day strategy result.
- A small positive net profit can still be highly uncertain once a wide Monte Carlo distribution is taken into account — here, nearly half of simulations finished below zero.

Conclusion

This project builds a complete evolutionary research framework spanning strategy generation, backtesting, genetic optimisation, rolling and expanding walk-forward validation, overfitting diagnostics, trade-level analysis, Monte Carlo resampling, and automated experiment management.

The current experiment does not support a profitable-strategy claim: the best configuration produced only \$30 net profit over 135 trades, the Monte Carlo median net profit is \$15, and 48.7% of simulations finish below zero. Results are also unstable across GA repetitions. The framework was built precisely to make these weaknesses visible rather than to hide them.

Portfolio statement — This project demonstrates the ability to build research infrastructure, organise experiments rigorously, and read weak or ambiguous results critically rather than optimistically.

Skills demonstrated: Python engineering, evolutionary optimisation, time-series validation, statistical diagnostics, and reproducible research practice.